Deriving $c(x)$: the control-authority ceiling of an SSA safety constraint

Expanded edition, with a primer for readers light on calculus

SPARK goal-insertion notes

Notation primer (read this first if calculus is rusty)

This whole note rests on a handful of symbols. Here is each one in plain words.

A "vector" is just a list of numbers. The robot's pose is a list of joint angles, written $x = (x_1, \dots, x_n)$. The control is another list $u = (u_1, \dots, u_m)$: one number per joint saying how fast to move that joint right now. We treat lists as points/arrows in space.

A derivative is a rate of change — a slope. If a quantity q depends on a knob s, the derivative $\dfrac{dq}{ds}$ answers: if I nudge s up by a tiny amount, how much does q change, per unit of nudge? Positive = q goes up, negative = q goes down, big = steep.

A dot over a letter means "rate of change in time." $\dot{x}$ ("x-dot") is the velocity of the pose: how fast each joint angle is changing per second. By the robot's dynamics, $\dot{x}$ is exactly the control u (you command velocities). Likewise $\dot{\phi}$ ("phi-dot") is how fast the safety number ϕ is changing.

A partial derivative $\dfrac{\partial \phi}{\partial x_j}$ is a slope in one direction. ϕ depends on the whole pose x. Freeze every joint except joint j, wiggle only x_j, and ask how fast ϕ changes. That single slope is $\partial\phi/\partial x_j$. There are n of them, one per joint.

The symbol $\nabla\phi$ ("nabla phi", or "the gradient of ϕ") collects all those slopes into one list:

$$\nabla\phi = \left(\frac{\partial\phi}{\partial x_1}, \frac{\partial\phi}{\partial x_2}, \dots, \frac{\partial\phi}{\partial x_n}\right).$$

Picture ϕ as the height of a hill over the field of poses. $\nabla\phi$ is the arrow that points straight uphill, and its length says how steep the hill is. That is the only thing ∇ means here: "the uphill arrow / the list of slopes." $\nabla\phi^\top$ (with the little $\top$, "transpose") is the same list written as a row so we can multiply it — see the dot product next.

A dot product $a^\top b = a \cdot b = \sum_k a_k b_k$ measures alignment. Take two lists of equal length, multiply them entry by entry, add up. If b is a direction you move and $a = \nabla\phi$ is the uphill arrow, then $a \cdot b$ tells you how much you go uphill by moving along b: positive if b heads uphill, negative if downhill, zero if you move across the slope (sideways, no height change).

$|y|$ is absolute value (drop the sign: $|-3| = 3$). sign(y) is $+1$ if $y > 0$, -1 if $y < 0$.

$\min_{u \in \mathcal{U}}(\text{something})$ means "the smallest value of something as u ranges over all allowed controls $\mathcal{U}$." Here $\mathcal{U}$ is the actuator box: each joint speed u_k is capped, $-u_{lim,k} \le u_k \le u_{lim,k}$. So the "box" is literally a box of allowed control vectors, and "minimize over the box" = "pick the allowed control that makes the quantity as small (negative) as possible." (max is the same idea for the largest value; $\exists$ means "there exists.")

With that vocabulary, the rest is bookkeeping. The one calculus fact we use is the chain rule, explained in Step 1.

Setup and notation

symbol	meaning
$x \in \mathbb{R}^n$	robot configuration (list of n joint positions; the safety-index state)
$u \in \mathbb{R}^m$	control input (list of m joint velocities we command)
$\mathcal{U} = \{u : \lvert u_k \rvert \le u_{lim,k}\}$	actuator box ($= \prod_k [-u_{lim,k},\ u_{lim,k}]$)
$\dot{x} = f(x) + G(x)\,u$	dynamics; here $f \equiv 0$, $G = I$, so simply $\dot{x} = u$
$\phi(x)$	safety index (one constraint); safe $\{\phi \le 0\}$, danger $\{\phi > 0\}$
$\eta > 0$	SSA margin rate: enforce $\dot{\phi} \le -\eta$ when $\phi \ge 0$
n	unit contact normal (the distance-decreasing / escape direction)
$J(x)$	Jacobian of the contacting body point, $\mathbb{R}^{3\times n}$ (defined in Step 1)

Our goal: derive $c(x)$, the maximum rate at which the actuators can drive ϕ down, and show the single-active-constraint QP is feasible iff (if and only if) $\eta \le c(x)$ (for first-order dynamics).

Why care? ϕ going up means heading into collision. The safety filter's whole job is to force ϕ down fast enough ($\dot{\phi} \le -\eta$). $c(x)$ is the most "down" the motors can manage. If even that is not enough, the filter is beaten — and $c(x)$ tells us exactly when, with one number.

Step 1 — the time derivative of the safety index (the chain rule)

We want $\dot{\phi}$: how fast the safety number changes as the robot moves. ϕ depends on the pose x, and x changes in time at rate $\dot{x}$. The chain rule stitches these together. Intuition: if moving joint j a little changes ϕ at slope $\partial\phi/\partial x_j$, and joint j is currently moving at speed $\dot{x}_j$, then joint j's

contribution to $\dot{\phi}$ is (slope) $\times$ (speed) $= \dfrac{\partial \phi}{\partial x_j}\,\dot{x}_j$. Add up the contributions of all joints:

$$\dot{\phi} \;=\; \sum_{j=1}^{n} \frac{\partial \phi}{\partial x_j}\,\dot{x}_j \;=\; \nabla\phi(x)^{\top}\dot{x}.$$

That last equality is just the sum rewritten as a dot product of the uphill arrow $\nabla\phi$ with the velocity $\dot{x}$ — "how fast does ϕ climb, given which way and how fast we're moving."

Now substitute the dynamics $\dot{x} = f(x) + G(x)u$ and split the two pieces:

$$\dot{\phi} = \nabla\phi^{\top}\big(f(x) + G(x)u\big) = \underbrace{\nabla\phi^{\top} f}_{=:\,L_f\phi} + \underbrace{\nabla\phi^{\top} G}_{=:\,L_g\phi}\; u.$$

So $\dot{\phi} = L_f\phi + L_g\phi\, u$. The two named pieces:

- $L_f\phi$ — the drift: how ϕ would change on its own even with zero control ($u = 0$). It is a single number. For our robot $f \equiv 0$, so $L_f\phi = 0$ (nothing moves if we command nothing).
- $L_g\phi \in \mathbb{R}^{1\times m}$ — the control gain: a list with one entry per joint, saying how strongly that joint's velocity pushes ϕ. This is the knob the controller actually turns.

(The names $L_f\phi, L_g\phi$ are "Lie derivatives" — don't let the term scare you; they are just these two pieces of $\dot{\phi}$.)

What is $J(x)$ and where did the normal n go? For a collision index, ϕ depends on x only through the position $p(x)$ of the contacting body point (the spot on the arm nearest the obstacle). Two more chain-rule facts:

- Moving in Cartesian space, ϕ's uphill direction in 3D is the contact normal n (the straight line from obstacle center to the body point — the escape direction). So the 3D gradient of ϕ is n.
- The Jacobian $J(x)$ is the table that converts "joint velocities" into "how the contact point moves in 3D": $\dot{p} = J(x)\,\dot{x}$. Each column says where the point goes if you spin one joint.

Chain rule once more (3D slope n, times the joint$→$3D map J) gives the configuration-space gradient $\nabla_x\phi = J(x)^{\top}n$, and therefore

$$\boxed{L_g\phi(x) = n^{\top}J(x)\,G(x)} \qquad (\in \mathbb{R}^{1\times m}).$$

Read it right to left: G turns a command into a joint velocity, J turns that into a 3D motion of the contact point, and $n^{\top}(\cdot)$ reads off how much of that 3D motion is along the escape direction — i.e. how fast the command changes ϕ.

Step 2 — feasibility of the safety constraint

The SSA QP must find a control inside the box that makes ϕ recede fast enough:

$$\exists\, u \in \mathcal{U} : \quad L_f\phi + L_g\phi\, u \le -\eta.$$

(The inequality is $\dot{\phi} \le -\eta$, written out.)

When does such a u exist? Exactly when the best the box can do already meets the target. "Best" = most negative $\dot{\phi}$ (steepest descent of ϕ). So:

$$\min_{u \in \mathcal{U}} \left(L_f\phi + L_g\phi\, u \right) \;\le\; -\eta \quad \Longleftrightarrow \quad L_f\phi + \min_{u \in \mathcal{U}} L_g\phi\, u \;\le\; -\eta.$$

($L_f\phi$ does not depend on u, so it slides outside the min.) Now we just need that one minimization.

Step 3 — minimizing a linear form over the box

We must minimize $L_g\phi\, u = \sum_k (L_g\phi)_k\, u_k$ over the box. The happy fact: this sum is separable — joint k's term $(L_g\phi)_k\, u_k$ involves only u_k, and the box treats each u_k independently. So minimize each term on its own.

For one term, to make $(L_g\phi)_k\, u_k$ as negative as possible, push u_k to whichever end of its allowed range $[-u_{lim,k}, u_{lim,k}]$ opposes the sign of the gain:

$$u_k^\star = -\,\mathrm{sign}\big((L_g\phi)_k\big)\, u_{lim,k} \quad \Longrightarrow \quad (L_g\phi)_k\, u_k^\star = -\,\big|(L_g\phi)_k\big|\, u_{lim,k}.$$

(If the gain is positive, drive the joint full negative; if negative, full positive — either way the product comes out $-|\text{gain}| \times |\text{limit}|$, the most negative possible.) Summing the best term-by-term:

$$\min_{u \in \mathcal{U}} L_g\phi\, u \;=\; -\sum_{k=1}^{m} u_{lim,k}\, \big|(L_g\phi)_k\big|.$$

Step 4 — define $c(x)$

Flip the sign to talk about the maximum decrease rate, and name it:

$$c(x) \;:=\; \max_{u \in \mathcal{U}} \big(-L_g\phi\, u \big) \;=\; -\min_{u \in \mathcal{U}} L_g\phi\, u \;=\; \sum_{k=1}^{m} u_{lim,k}\, \big|(L_g\phi)_k\big|.$$

In words: $c(x)$ is the fastest the motors can drive the safety number down, found by giving every joint its full allowed speed in the most helpful direction. It is a sum of non-negative pieces (each = joint speed limit $\times$ that joint's push on ϕ), so $c(x) \ge 0$ always.

Plug into Step 2's feasibility test ($L_f\phi + \min(\cdots) \le -\eta$ becomes $L_f\phi - c(x) \le -\eta$):

$$L_f\phi - c(x) \le -\eta \quad \Longleftrightarrow \quad \boxed{c(x) \;\ge\; \eta + L_f\phi}.$$

The filter can keep up iff its maximum braking $c(x)$ is at least the demanded braking η plus whatever the drift $L_f\phi$ is already adding.

First-order / velocity control ($f \equiv 0 \Rightarrow L_f\phi = 0$, our case):

$$\boxed{\text{feasible} \;\Longleftrightarrow\; c(x) \ge \eta\,, \qquad c(x) = \sum_k u_{lim,k}\, \big|(L_g\phi)_k\big| = \big\|\,\mathrm{diag}(u_{lim})\, L_g\phi^\top\big\|_1.}$$

(The last form, $\|\text{diag}(u_{lim})L_g\phi^\top\|_1$, is the same sum written compactly: scale each gain by its limit, then take the ℓ_1 norm = "add up absolute values." Just notation.)

Step 3′ — asymmetric actuator box ($u_{\text{min}} \neq -u_{\text{max}}$)

Steps 3–4 assumed a symmetric box $[-u_{lim,k},\, u_{lim,k}]$. Real actuators are often asymmetric — a joint that can swing faster one way than the other, or a one-directional drive — so the allowed range is $u_k \in [u_{min,k},\, u_{max,k}]$ with $u_{min,k} \neq -u_{max,k}$. Nothing in the logic breaks; only the per-coordinate minimization of Step 3 changes, because the most-negative endpoint is no longer always $\mp u_{lim,k}$.

Write $g_k := (L_g\phi)_k$. A linear term $g_k u_k$ on an interval is minimized at an endpoint, chosen by the sign of g_k:

$$\min_{u_k\in[u_{min,k},u_{max,k}]} g_k u_k = \begin{cases} g_k\, u_{min,k}, & g_k > 0 \quad (\text{push the joint to its low end}), \\ g_k\, u_{max,k}, & g_k < 0 \quad (\text{push it to its high end}). \end{cases}$$

Taking $c(x) = -\min_u L_g\phi\, u$ term by term,

$$\boxed{c(x) = \sum_{k=1}^{m} \max\big(-g_k\, u_{min,k},\, -g_k\, u_{max,k}\big)}.$$

For each joint, pick whichever endpoint drives ϕ down hardest.

A cleaner reading: "spread − bias." Split each interval into its center and half-width,

$$u_{c,k} = \tfrac{u_{max,k}+u_{min,k}}{2}, \qquad r_k = \tfrac{u_{max,k}-u_{min,k}}{2},$$

so $u_k = u_{c,k} + \delta_k$ with $\delta_k \in [-r_k, r_k]$ symmetric. Then $L_g\phi\, u = L_g\phi\, u_c + L_g\phi\, \delta$, and the symmetric Step-3 result applies to the δ part:

$$c(x) = \underbrace{\sum_k r_k\, |g_k|}_{\text{spread (symmetric form, half-widths)}} - \underbrace{L_g\phi \cdot u_c}_{\text{bias = } \dot\phi \text{ at the box center}}.$$

The spread is exactly the old formula with $u_{lim,k} \to r_k$; the bias is the $\dot\phi$ the actuator already produces sitting at the box center u_c. Symmetric box $\Rightarrow u_c = 0, r_k = u_{lim,k}$, the bias vanishes, and we recover $c(x) = \sum_k u_{lim,k}|g_k|$.

What asymmetry does physically. The reachable $\dot\phi$ is no longer the centered interval $[-c, +c]$; it slides off-center by the bias:

$$\dot\phi \in \Big[\, L_g\phi \cdot u_c - \textstyle\sum_k r_k|g_k|,\;\; L_g\phi \cdot u_c + \sum_k r_k|g_k| \,\Big].$$

The half-width (braking range) is unchanged, but max-decrease and max-increase rates are now unequal — an asymmetric actuator is better at pushing ϕ one way than the other. The feasibility test keeps its form with this generalized c: feasible $\iff c(x) \geq \eta + L_f\phi$.

Implementation note. SPARK's box is symmetric by construction (`value_based_safe_algo.py`: $u_{\text{min}} = -\text{ControlLimit}$, $u_{\text{max}} = +\text{ControlLimit}$), so the symmetric formula — and `authority.py`'s `|Lg| @ u_lim` — is exact here. The QP solver (OSQP) takes the two bounds independently, so it would handle an asymmetric box natively; only the closed-form $c(x)$ needs the max-of-endpoints version above.

Step 5 — closed form in kinematic terms

Insert $L_g\phi = n^\top J(x)\, G(x)$ from Step 1:

$$c(x) \;=\; \sum_{k=1}^{m} u_{lim,k} \left| \left[\, n^\top J(x)\, G(x) \,\right]_k \right| \;=\; \left\| \text{diag}(u_{lim})\, \left(n^\top J(x) G(x)\right)^\top \right\|_1 .$$

It is the actuator-limit-weighted sum of how strongly each joint can push the contact point along the escape direction — pure geometry of the current pose plus the speed limits. No simulation, no tuning.

Step 6 — geometric reading (the reachable-velocity set)

Here is the picture behind $c(x)$. As the control u ranges over its box, the contact point's 3D velocity $v = J(x)G(x)\,u$ sweeps out a solid shape $\mathcal{Z}$ (a "zonotope" — think of a squashed box of all reachable point-velocities). Since $\dot\phi = n^\top v$ (how fast ϕ climbs = component of v along the escape direction n),

$$c(x) \;=\; \max_{v \in \mathcal{Z}} \left(-\, n^\top v \right)$$

— how far the reachable-velocity shape sticks out in the escape direction $-n$. If the shape reaches far along $-n$, the body can retreat fast (c large); if it barely reaches that way, the body can hardly retreat (c small). The reachable $\dot\phi$ values form exactly the symmetric interval $[-c(x),\, +c(x)]$, so $c(x)$ is its half-width: the most ϕ can be pushed down — and, equally, the most an adversary could force it up.

Step 7 — special cases

- Round speed limit instead of a box ($\|u\|_2 \le r$): the same derivation with a ball gives $c(x) = r\,\|L_g\phi\|_2$ (an ℓ_2 norm instead of the weighted ℓ_1). Same gradient, different limit-shape.
- Far joints don't matter: $J(x)$ has zero columns for joints beyond the contacting body (spinning your wrist can't move your shoulder's contact point), so those $(L_g\phi)_k = 0$. Only joints between the base and the contact contribute to $c(x)$.

Step 8 — the degenerate case $c(x) = 0$

Since $c(x)$ is a sum of non-negative terms, it is zero only when every term is zero:

$$c(x) = 0 \iff L_g\phi = n^\top J(x)G(x) = \mathbf{0} \iff n \perp \text{Im}\big(J(x)G(x)\big).$$

Meaning: the escape direction n is perpendicular to every motion the joints can produce at the contact point. The body can slide along the obstacle surface but cannot move away from it at all — a controllability singularity of the constraint. There, no $\eta > 0$ is feasible: the filter has zero braking authority no matter how mild the demand. (This is the extreme end of "actuator can't keep up.")

Step 9 — extension to multiple active constraints

So far: one obstacle/contact. With several active at once (indices $i \in \mathcal{A}$), write $a_i := L_g\phi_i^\top$ (constraint i's gain list) and $\beta_i := \eta_i + L_f\phi_i$ (its demand). Now we need one u in the box that satisfies all the rows simultaneously. Feasibility becomes

$$\mu(x) := \min_{u\in\mathcal{U}} \max_{i\in\mathcal{A}} \left(a_i^\top u + \beta_i\right) \le 0.$$

Read the inner max as "the worst (least-satisfied) constraint at this u"; the outer min picks the u that makes that worst case as good as possible. If even the best worst-case is still ≤ 0, all rows hold — feasible. This is a small linear program (LP); no single closed form, because the joints can no longer be optimized one at a time (one u must serve several rows that may pull different ways).

Dual / aggregated form. Lifting the max to a weight y over the simplex $\Delta = \{y \ge 0,\ \sum_i y_i = 1\}$ and swapping min / max (a standard saddle-point move) gives

$$\mu(x) = \max_{y\in\Delta} \Big[\, y^\top\beta - \underbrace{\textstyle\sum_k u_{lim,k}\big|\big(\sum_i y_i a_i\big)_k\big|}_{=:\ c_y(x)} \Big].$$

The bracketed sum $c_y(x)$ is exactly the single-constraint ceiling $c(x)$ of Step 4, but applied to a blended normal $\bar{a}(y) = \sum_i y_i\, L_g\phi_i$. Therefore:

$$\boxed{\text{feasible} \iff \forall\, y \in \Delta:\ c_y(x) \ \ge\ \sum_i y_i(\eta_i + L_f\phi_i)\ .}$$

For every way of blending the active constraints, the blended braking authority must still beat the blended demand. Infeasibility needs just one bad blend $y^\star$:

$$\exists\, y^\star \ge 0:\quad c_{y^\star}(x) \ <\ \sum_i y_i^\star(\eta_i + L_f\phi_i).$$

The worst case is the pincer: pick $y^\star$ so the normals nearly cancel, $\bar{a}(y^\star) \approx \mathbf{0}$ (so $c_{y^\star} \approx 0$ — almost no usable braking in the blended direction), while the demands still add up positive. In the limit this is the Farkas certificate "$\sum_i y_i L_g\phi_i = \mathbf{0}$ and $\sum_i y_i(\eta_i + L_f\phi_i) < 0$": obstacle A wants me to go $+x$, obstacle B wants $-x$, and I must retreat from both at once — impossible.

What changed vs. one constraint.

- One active constraint: $\Delta = \{1\}$, $\bar{a} = a_1$, and the box recovers $c(x) \ge \eta + L_f\phi$ (Step 4).
- Checking each constraint alone is necessary but not sufficient: $\eta_i \le c_i(x)$ for every i only covers the corners $y = e_i$; a mixed blend $y^\star$ can still fail. Conflict lives in the interior of Δ — which is why no single scalar suffices and we solve the LP.

Summary

$$c(x) = \max_{u \in \mathcal{U}} \left(-L_g \phi \, u \right) = \sum_k u_{lim,k} \left| \left[n^\top J(x) G(x) \right]_k \right|$$

$$\text{feasible} \iff c(x) \geq \eta + L_f \phi$$

- $c(x)$ is the best achievable rate of decrease of the safety index under the actuator limits — the single number that says whether the safety filter can keep up.
- It depends only on the pose (through J, G) and the contact geometry (through n) — not on velocity, for first-order dynamics.
- An attacker steers the closed loop to a boundary contact that minimizes $c(x)$; $c(x) \to 0$ at singularities, so lowering η shrinks but never closes the vulnerable set.